# Language Project Ideas

Forty project choices grounded in the Python interpreter. The intended minimum is approximately four to eight hours for someone familiar with the interpreter, including their testing and documentation. These are planning estimates, not measured guarantees. The debugger, VM, and runtime-representation projects may need reduced scope to meet that target.

Students must submit convincing evidence: a runnable test suite or test set, expected results, a documented command, and checks for normal behavior, boundaries, failures, and interactions with existing features. Students write the tests. These notes describe the projects and suggested evidence, not test implementations.

All snippets describe proposed syntax. Assertions already exist and are excluded as a new feature. Constant bindings and immutable collections are separate choices. Both the REPL and operating-system arguments remain separate choices. Try-catch uses `try block catch identifier block`, with an error value/status propagated to a handler or program boundary, without an error-type system.

## 1. Compound assignment

Estimated total: 4-6 hours

```
items[nextIndex()] += 1;
```

- **Implementation:** Add += and related tokens, assignment grammar, and an evaluator path that resolves a target once.
- **Design:** Preserve local assignment rules. Resolve the base and index once before reading, calculating, and writing.
- **Test evidence:** Demonstrate a side-effecting index function, chained fields, invalid targets, and a failed RHS.

**Minimum scope and implementation notes:** Use parse_expression and the assign branch as the starting points. Rewriting a[i()] += v as a[i()] = a[i()] + v would evaluate i twice. Decide whether set needs its own update form. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 2. Range expressions

Estimated total: 4-6 hours

```
values = range(2, 10, 2);
```

- **Implementation:** Add a dedicated range expression and construct a list of integers.
- **Design:** Define an exclusive stop, positive and negative steps, and rejection of zero steps.
- **Test evidence:** Cover empty ranges, descending ranges, boundary endpoints, and invalid types.

**Minimum scope and implementation notes:** Minimum: require all three arguments and integer inputs, excluding booleans. Reuse the dedicated built-in parsing style. Demonstrate [2, 4, 6, 8] for the example. Specify allocation limits or document large-range behavior. Lazy iterators and for-loop syntax are outside scope. Estimated total includes the student's runnable tests and explanation.

## 3. Exponentiation and remainder

Estimated total: 4-6 hours

```
print(2 ** 3 ** 2); print(17 % 5);
```

- **Implementation:** Extend token ordering, precedence rules, and numeric evaluation.
- **Design:** Define exponent associativity and unary-minus precedence. Specify remainder signs and zero-divisor behavior.
- **Test evidence:** Demonstrate the chosen result for -2 ** 2, fractional operands, and booleans.

**Minimum scope and implementation notes:** Treat the two numeric operators as one coherent numeric-semantics project. A single Python operator dispatch is too small by itself. Require a grammar change, edge-case specification, and explanations of differences from host Python. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 4. For-each loops

Estimated total: 6-8 hours

```
for (item in items) { print(item) };
```

- **Implementation:** Add for/in tokens, a loop AST, and iteration evaluation.
- **Design:** Define loop-variable scope, collection mutation behavior, and handling of break, continue, return, and exit.
- **Test evidence:** Demonstrate empty collections, nested loops, and a closure capturing the loop variable.

**Minimum scope and implementation notes:** Choose list and string iteration as the minimum. Objects are optional. Specify whether each iteration creates a fresh binding, and whether traversal uses a snapshot or a live collection. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 5. C-style for loops

Estimated total: 6-8 hours

```
for (i = 0; i < 10; i = i + 1) { print(i) };
```

- **Implementation:** Parse initializer, condition, update, and body. Extend the evaluator or lower to a careful core form.
- **Design:** A continue must reach the update. Return and exit must escape without running the update.
- **Test evidence:** Demonstrate continue in a nested conditional and break in a nested loop.

**Minimum scope and implementation notes:** Minimum: require all three header clauses. Use existing assignment syntax, with no dependency on compound assignment. A continue must execute the update. Define variable visibility and status propagation. Omitted clauses are optional. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 6. Destructuring assignment

Estimated total: 6-8 hours

```
[first, second] = pair;
```

- **Implementation:** Add assignment-pattern parsing and recursive binding or target validation.
- **Design:** Evaluate the RHS once. Define length mismatch and whether a failed match leaves partial assignments.
- **Test evidence:** Demonstrate nested patterns, mismatched shapes, and repeated target names.

**Minimum scope and implementation notes:** Minimum: one flat, exact-length list pattern containing distinct identifiers. Nested patterns, indexed targets, and repeated names are optional or rejected. Evaluate the RHS once and validate shape before binding, so mismatch causes no partial writes. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 7. Default parameters

Estimated total: 6-8 hours

```
function greet(name, prefix = "Hello") { return prefix + name };
```

- **Implementation:** Extend parameter records, function values, and argument binding in call evaluation.
- **Design:** Choose when defaults evaluate and which environment they use. Distinguish omitted arguments from explicit null.

- **Test evidence:** Demonstrate captured defaults, mutable defaults, and too many arguments.

**Minimum scope and implementation notes:** The current function value stores parameter names and its defining environment. Definition-time and call-time defaults are both plausible designs, but students must select and explain one. Require an example where the choices differ. Minimum: trailing defaulted parameters only. Choose a single default-evaluation rule and document it. Keyword arguments are outside scope. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 8. Keyword arguments

Estimated total: 6-8 hours

```
result = divide(denominator: 2, numerator: 10);
```

- **Implementation:** Extend call-argument syntax and bind arguments by parameter name.
- **Design:** Define positional/keyword ordering. Reject duplicates, unknown names, and missing required parameters.
- **Test evidence:** Demonstrate reordered arguments and side effects that reveal evaluation order.

**Minimum scope and implementation notes:** The colon spelling avoids confusing named arguments with the existing assignment expression. Default parameters are optional, not a prerequisite. Evaluate supplied arguments in source order before binding. Minimum: fixed parameters with required values. Default parameters and keyword spread are outside scope. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 9. Rest parameters

Estimated total: 6-8 hours

```
function count(...items) { return length(items) };
```

- **Implementation:** Add a final rest parameter to function syntax and collect extra arguments into a list.
- **Design:** Allow one rest parameter at the end. Keep ordinary fixed-arity functions unchanged.
- **Test evidence:** Cover zero extra arguments, several extras, too few fixed arguments, and invalid parameter placement.

**Minimum scope and implementation notes:** Minimum: rest parameters only. Call-site spread is outside scope. Update function parameter metadata and the call branch, preserving left-to-right argument evaluation and closure environment links. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 10. Repeat-until loops

Estimated total: 4-6 hours

```
repeat { set n = n - 1 } until (n == 0);
```

- **Implementation:** Add repeat/until tokens, a body-first loop node, and evaluator support.
- **Design:** Execute the body once before checking a boolean condition. Continue goes to the condition.
- **Test evidence:** Cover one execution, repeated execution, break, continue, and return through a function.

**Minimum scope and implementation notes:** Minimum: brace-delimited body with a parenthesized condition. Reuse the while branch's status conventions without consuming return or exit. Invalid non-boolean conditions must produce a language error. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 11. Switch with literal cases

Estimated total: 6-8 hours

```
switch (code) { case 1: { print("one") } default: { print("other") } };
```

- **Implementation:** Add switch/case/default grammar and a selector AST node.
- **Design:** Evaluate the selector once. Choose one literal case with no fall-through, using language equality.
- **Test evidence:** Cover a match, default, no match, duplicate cases, and a side-effecting selector.

**Minimum scope and implementation notes:** Minimum: number, string, and boolean literal labels with braced bodies. Explicitly exclude guards, patterns, and fall-through. Pass existing loop break/continue statuses outward rather than giving switch a second meaning for break. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 12. List comprehensions

Estimated total: 6-8 hours

```
squares = [x * x for x in values];
```

- **Implementation:** Parse a single comprehension clause and build a result list in the evaluator.
- **Design:** Evaluate the source once. Use a temporary loop binding that does not leak.
- **Test evidence:** Cover empty input, outer-name lookup, a same-named outer variable, and invalid source types.

**Minimum scope and implementation notes:** Minimum: one loop over a list, with no filters or nested clauses. An optional filter is a stretch goal. Define the temporary binding's lifetime and preserve return/exit propagation from called expressions. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 13. Membership operator

Estimated total: 4-6 hours

```
print("name" in record); print(3 in numbers);
```

- **Implementation:** Add an in operator at a specified comparison precedence and evaluate by collection type.
- **Design:** Lists use vertex_equal, objects check string keys, and strings check substrings.
- **Test evidence:** Cover nested list values, true versus 1, missing keys, empty strings, and invalid operand types.

**Minimum scope and implementation notes:** Minimum: one in operator, without not in or chained membership. Use the existing structural equality helper for lists, including cyclic data. Reject function dictionaries as ordinary objects. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 14. Negative indexing

Estimated total: 4-6 hours

```
print(items[-1]); items[-2] = 7;
```

- **Implementation:** Extend require_index so negative integers count from the end for reads and list writes.
- **Design:** Keep integer validation and boolean rejection. Empty and out-of-range access still fails.
- **Test evidence:** Cover -1, -length, too-negative indices, empty inputs, strings, and list mutation.

**Minimum scope and implementation notes:** Minimum: indexing only. Existing slice() bounds remain unchanged and that distinction must be documented. Because read and write paths share require_index, explain why both gain the same behavior. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 15. Bracket slice syntax

Estimated total: 6-8 hours

```
part = items[1:4]; prefix = text[:3];
```

- **Implementation:** Extend postfix parsing to distinguish index and slice suffixes, then reuse slice-bound rules.
- **Design:** Use a half-open range. Omitted start is zero and omitted end is length.
- **Test evidence:** Cover lists, strings, omitted bounds, empty slices, invalid bounds, and chained suffixes.

**Minimum scope and implementation notes:** Minimum: two-bound read-only slices with the current nonnegative bound rules. Exclude step sizes and slice assignment. Retain the existing slice() function-like form and its shallow-copy semantics. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 16. Chained comparisons

Estimated total: 6-8 hours

```
print(0 < nextValue() < 10);
```

- **Implementation:** Parse a comparison chain and evaluate adjacent pairs using shared comparison rules.
- **Design:** Evaluate each middle operand once. Stop before evaluating later operands when a comparison fails.
- **Test evidence:** Cover mixed comparison operators, a side-effecting middle expression, and skipped later calls.

**Minimum scope and implementation notes:** Minimum: chains of existing comparison operators. Extract or share the evaluator’s existing pairwise comparison semantics rather than using Python truth coercion. Specify how equality operators can appear in a chain. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 17. Named string interpolation

Estimated total: 6-8 hours

```
message = "Hello, ${name}!";
```

- **Implementation:** Scan interpolation markers in strings and represent literal segments and name lookups.
- **Design:** Resolve names in the current lexical environment and convert with language string rules.
- **Test evidence:** Cover several names, outer bindings, escaped markers, unknown names, and malformed markers.

**Minimum scope and implementation notes:** Minimum: identifier substitutions only, with a documented escape rule. Arbitrary expressions and nested interpolation are outside scope. Preserve existing string escape behavior and source-location diagnostics. Never use Python eval. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 18. Null-coalescing operator

Estimated total: 4-6 hours

```
label = supplied ?? "untitled";
```

- **Implementation:** Add a ?? token, a precedence rule, and a lazy binary AST node.
- **Design:** Evaluate the right side only when the left value is null. False, zero, and an empty string remain values.
- **Test evidence:** Cover null and non-null operands, side effects, chaining, and precedence with assignment.

**Minimum scope and implementation notes:** Minimum: null coalescing only, without optional field access or coalescing assignment. Specify associativity and how a non-normal status from either evaluated operand propagates. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 19. Object-field deletion

Estimated total: 4-6 hours

```
delete record["temporary"];
```

- **Implementation:** Add delete syntax for an object field target and evaluator support.
- **Design:** Evaluate the object and key once. Specify missing-key behavior and reject lists and function internals.
- **Test evidence:** Cover existing and missing keys, aliases to the same object, invalid targets, and side-effecting keys.

**Minimum scope and implementation notes:** Minimum: deletion as a statement, supporting existing bracket and dot field forms. Environment bindings cannot be deleted. Use the existing complex target AST but reject a bare identifier or call target. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 20. Object key and value enumeration

Estimated total: 4-6 hours

```
names = keys(record); contents = values(record);
```

- **Implementation:** Add dedicated keys and values expression forms using the existing built-in parsing pattern.
- **Design:** Return new lists in a documented field order. Specify whether values retain references to nested objects.
- **Test evidence:** Cover empty objects, insertion order, nested aliases, and rejection of functions and non-objects.

**Minimum scope and implementation notes:** Minimum: two enumeration operations with explicit snapshot and aliasing rules. keys and values must reject interpreter function dictionaries and must never expose closure environments or reserved runtime metadata. Estimate assumes familiarity with the interpreter and includes a student-written runnable test set plus a short explanation. Require expected results, failure status, and regression evidence. This slide describes a proposal, not an implemented feature.

## 21. Lexical block scope

Estimated total: 6-8 hours

```
if (true) { x = 2; set total = total + x };
```

- **Implementation:** Give control-structure blocks their own environment linked to the enclosing environment.
- **Design:** Ordinary assignment writes locally. set updates the nearest existing binding. Specify whether each loop iteration creates a fresh environment.
- **Test evidence:** Demonstrate shadowing, an undefined set target, nested blocks, and closures that retain block bindings.

**Minimum scope and implementation notes:** Minimum: block scope only. Do not require declaration syntax or constants. The existing evaluator already uses parent-linked environments. This changes current behavior, so explain compatibility effects and how return/break/continue/exit propagate. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 22. Try-catch with an error status

Estimated total: 4-8 hours

```
try { throw "problem" } catch error { print(error) };
```

- **Implementation:** Parse try block catch identifier block as one statement. Add throw and an error value/status pair.

- **Design:** Propagate the error status until a catch consumes it. Catch creates the named variable with the error value and evaluates its block.
- **Test evidence:** Demonstrate nested handlers, propagation through a call and loop, an uncaught error reaching the program boundary, and unchanged return behavior.

**Minimum scope and implementation notes:** No parentheses around the catch identifier. No error type system. Minimum: explicit throw of a language value, one handler, and a clear uncaught-error report/nonzero exit at the runner. Converting existing runtime failures is an optional extension with an explicitly bounded list. No finally requirement. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 23. Constant bindings

Estimated total: 4-8 hours

```
const limit = 10;
```

- **Implementation:** Add const declarations and binding metadata used by ordinary assignment and set.
- **Design:** Initialize once and reject rebinding. Keep constant binding rules separate from collection mutability.
- **Test evidence:** Demonstrate reassignment, set through an enclosing environment, legal shadowing, and a constant name referring to a mutable list.

**Minimum scope and implementation notes:** Minimum: constant bindings in the existing lexical environments. Metadata must not collide with user object fields. Const does not freeze a collection. Immutable collections are a separate project and the combination is optional. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 24. An immutable list type

Estimated total: 6-8 hours

```
items = immutable([1, 2, 3]);
```

- **Implementation:** Add a distinct immutable list runtime type and update indexing, type reporting, equality, printing, and copy behavior.
- **Design:** Mutation restrictions belong to the collection type and apply through every alias.
- **Test evidence:** Demonstrate alias-based mutation rejection, valid reads, equality, type reporting, and nested-value behavior.

**Minimum scope and implementation notes:** Minimum: immutable lists of scalar values, rejecting nested mutable collections at construction. This bounds deep-freezing and cycle complexity. Binding immutability is independent: a variable may be rebound to a different immutable list. Nested immutable structures are optional. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 25. A statement debugger

Estimated total: 6-8 hours

```
debug program.v
```

- **Implementation:** Preserve source locations on executable statement nodes and add a pause hook before statement execution.
- **Design:** Support step, continue, line breakpoints, and read-only variable inspection through lexical environments.
- **Test evidence:** Supply scripted command sessions checking stop locations, visible shadowed names, repeated loop stops, and normal completion.

**Minimum scope and implementation notes:** Minimum: terminal debugger for one source file, statement step-into only. A hook must run at statement boundaries, not on every expression recursion. Document treatment of several statements on one line. Step-over, step-out, variable editing, and graphical UI are optional. Estimated timing assumes familiarity with evaluator instrumentation. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 26. Coverage for language programs

Estimated total: 4-8 hours

```
coverage program.v
```

- **Implementation:** Assign source locations and stable IDs to executable statement nodes. Record executed IDs and generate a text report.
- **Design:** Compute the denominator from all executable statements, including uncalled function bodies. Distinguish statements from lines.
- **Test evidence:** Demonstrate untaken branches, uncalled functions, loops, blank/comment lines, and several statements sharing a line.

**Minimum scope and implementation notes:** This measures source programs in this language, not Python interpreter coverage. Minimum: one-file statement coverage with counts and uncovered locations. Count a loop statement once as covered even if it executes repeatedly. Branch coverage, merged runs, and HTML are optional. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 27. A bytecode compiler and stack VM

Estimated total: 6-8, ambitious hours

```
PUSH 2; PUSH 3; ADD; STORE x
```

- **Implementation:** Compile the supported AST subset into stack instructions and execute them in a small VM.
- **Design:** Define instruction effects, stack balance, and patched jump targets for conditionals and loops.
- **Test evidence:** Run equivalent programs through the tree interpreter and VM and compare outputs, variable values, and failures.

**Minimum scope and implementation notes:** Minimum subset: numbers, variables, arithmetic, print, if, and while. Explicitly reject unsupported nodes. Functions, closures, collections, break/continue, and full-language compatibility are outside scope. This is at the upper end of the estimate and may need a smaller subset if implemented without starter infrastructure. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 28. JSON data interchange

Estimated total: 4-8 hours

```
text = jsonEncode(record); value = jsonDecode(text);
```

- **Implementation:** Add encode/decode operations and map language data values to JSON.
- **Design:** Support null, booleans, finite numbers, strings, lists, and objects. Reject functions, cycles, and unsupported numbers.
- **Test evidence:** Demonstrate round trips, escaping, malformed JSON, boolean/number distinctions, cyclic data, and functions nested in collections.

**Minimum scope and implementation notes:** Use Python’s JSON parser/encoder with explicit language-value validation. No host eval. Runtime functions are dictionaries internally and must be detected before ordinary objects. Bounded scope excludes custom JSON parsing and object serialization protocols. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 29. A persistent REPL

Estimated total: 4-8 hours

```
Run the interpreter with no source argument
```

- **Implementation:** When no source file or inline source is supplied, enter an interactive loop. Reuse one environment across submissions so variables, functions, and closures persist.

- **Design:** Evaluate each submission once, display expression results, recover from language errors, and exit cleanly on EOF or an exit command.
- **Test evidence:** Supply scripted sessions demonstrating retained variables, a function defined and called later, recovery after an error, and clean exit.

**Minimum scope and implementation notes:** Minimum: one-line submissions with brace-delimited constructs on one line. Multiline input and continuation prompts are optional. The current runner requires one argument and has no REPL. Because a program/statement-list evaluation discards expression results, define how interactive expression results are obtained without evaluating twice. Keep source-file execution available. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 30. Operating-system arguments

Estimated total: 4-8 hours

```
vertex program.v first "two words"
```

- **Implementation:** Extend the runner to accept a source filename followed by zero or more arguments. Expose those arguments to the language as argv, a list of strings in the initial environment.
- **Design:** Exclude the source filename from argv. Preserve argument order and each shell-supplied string. No arguments gives an empty list.
- **Test evidence:** Launch the runner from tests with no arguments after the file, several strings, an empty string, spaces, and strings resembling numbers or flags.

**Minimum scope and implementation notes:** Minimum: argv is an ordinary global binding. Shell quoting has already been processed before Python receives sys.argv, so do not split strings again. Preserve existing inline-source use by documenting an explicit mode or unambiguous selection rule. REPL behavior is a separate project, not a prerequisite. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 31. Call-site argument spreading

Estimated total: 4-8 hours

```
result = add(...values);
```

- **Implementation:** Extend call argument ASTs and expand a list into positional arguments before function binding.
- **Design:** Evaluate arguments in source order once. Validate spread types and check arity after expansion.
- **Test evidence:** Demonstrate empty lists, several spreads, mixed arguments, side effects, and wrong argument counts.

**Minimum scope and implementation notes:** Minimum: list spreading in function calls only. Rest parameters, object spreading, and keyword spreading are separate features. Longer punctuation must precede the existing dot token. A bare two-parameter function is enough to demonstrate the extension. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 32. Bound methods and self

Estimated total: 6-8 hours

```
saved = counter.increment; saved();
```

- **Implementation:** Create a bound-method value when object field lookup retrieves a function. Carry the receiver into function-call binding.
- **Design:** Supply self while preserving the function's defining lexical environment. Extracted methods retain their receiver.
- **Test evidence:** Demonstrate direct calls, an extracted method, two receivers, and ordinary non-function fields.

**Minimum scope and implementation notes:** Minimum: object-held functions and an implicit self binding, without classes, inheritance, or constructors. Specify handling of a user parameter named self, equality, printing, and type reporting. A dedicated internal runtime representation avoids exposing interpreter dictionaries. Estimates are planning judgments for someone familiar with the interpreter and include student-written runnable tests and a short explanation. No test code is supplied in this deck.

## 33. Labeled loop control

Estimated total: 4-8 hours for the bounded minimum, including student-written tests and documentation.

Add labels to loops and allow break or continue to name an enclosing loop. Extend parsing and carry the target label with the control-flow status. Each loop consumes only statuses addressed to it. Minimum: labeled while loops within one function, with unknown labels and cross-function jumps rejected. Students supply cases for nested loops, continue targets, missing labels, and unchanged unlabeled behavior.

## 34. Static control-flow checker

Estimated total: 4-8 hours for the bounded minimum, including student-written tests and documentation.

Walk the AST before evaluation to reject return outside functions, break/continue outside loops, and duplicate function parameters. Track loop and function context explicitly, resetting loop context when entering a function. Minimum: these structural checks, without type inference or general undefined-name analysis. Students supply accepted and rejected programs, including a function defined inside a loop.

## 35. Source pretty-printer

Estimated total: 4-8 hours for the bounded minimum, including student-written tests and documentation.

Generate valid language source from the AST, preserving meaning through parentheses, string escaping, and statement separators. Minimum: the current Chapter 12 grammar, without comment or original whitespace preservation. Students supply parse-print-parse comparisons and behavioral examples. Source positions may be normalized when comparing ASTs.

## 36. Constant folding

Estimated total: 4-8 hours for the bounded minimum, including student-written tests and documentation.

Replace arithmetic subtrees containing numeric literals with equivalent literals. Preserve language type rules, evaluation order, and when errors become observable. Minimum: numeric literal arithmetic only, leaving divisions by zero and unsupported operations unchanged for runtime handling. Students compare original and transformed programs and show that calls, assignments, and unreachable errors are not evaluated during optimization.

## 37. Execution profiler

Estimated total: 4-8 hours for the bounded minimum, including student-written tests and documentation.

Instrument statement execution and function calls to report counts by stable source location and function identity. Minimum: counts, without wall-clock timing or a graphical report. Keep recursive calls and distinct closures identifiable, and prevent reporting from changing language output. Students supply programs whose counts can be calculated independently, including loops, recursion, and uncalled functions.

## 38. Optional field access

Estimated total: 4-8 hours for the bounded minimum, including student-written tests and documentation.

Add null-safe field access with a precise chain rule. A null receiver produces null without evaluating the remaining optional chain. Minimum: read-only field access chains, without assignment or optional function calls. Students supply non-null access, null at different chain positions, side-effecting skipped accesses where supported, and ordinary missing-field errors. Define whether grouping ends a chain.

## 39. Object spread

Estimated total: 4-8 hours for the bounded minimum, including student-written tests and documentation.

Extend object literals to combine existing objects and explicit fields, evaluating entries in source order. Minimum: shallow spread of objects, with a documented last-entry-wins rule and rejection of functions or other operands. Students supply overwrite-order cases, multiple spreads, empty objects, and examples showing that nested mutable values retain shared references.

## 40. Function composition

Estimated total: 4-8 hours for the bounded minimum, including student-written tests and documentation.

Add an operator that creates a unary function applying one function to another’s result. Preserve each function’s lexical environment, evaluate the operand expressions once, and propagate non-normal statuses. Minimum: unary composition only, with specified precedence and associativity. Students supply captured environments, chained compositions, side effects revealing call order, and non-function or wrong-arity failures.